Guide Conceptuel Complet du Parser : Tokens → AST

Version Détaillée et Exhaustive

Le problème fondamental

Tu as une **liste de tokens** générée par le lexer :

```
INT IDENTIFIER("main") LPAREN RPAREN LBRACE RETURN NUMBER(2) SEMICOLON
RBRACE
```

Mais ces tokens sont juste une **liste plate**. Tu dois :

1. **Comprendre la structure** qu'ils représentent (c'est une fonction qui retourne un nombre)
2. **Vérifier que c'est syntaxiquement correct** (les parenthèses correspondent, le SEMICOLON est présent, etc.)
3. **Organiser tout ça dans un AST hiérarchique** qui représente la structure du programme

Le **parser** est le programme qui fait **exactement ça** : il transforme une liste plate de tokens en une **structure arborescente (AST) bien organisée**.

Analogie du monde réel

Imagine que tu reçois une phrase : **"Le chat noir mange une souris"**

Les tokens seraient : `[ARTICLE, NOM, ADJECTIF, VERBE, ARTICLE, NOM]`

Le parser doit comprendre que c'est structuré comme :

```
PHRASE
├── SUJET
│   ├── ARTICLE : "Le"
│   ├── NOM : "chat"
│   └── ADJECTIF : "noir"
├── VERBE : "mange"
└── OBJET
    ├── ARTICLE : "une"
    └── NOM : "souris"
```

C'est exactement ce que le parser C fait : il transforme une liste plate en arbre structuré.

Concept 1 : Le curseur (Current Index)

Qu'est-ce que c'est exactement ?

Le **curseur** est un **index numérique** qui indique **quelle position tu es** dans la liste des tokens.

C'est comme un pointeur qui dit : "Tu lis le token numéro X".

Exemple visuel détaillé

Imagine que tu as les tokens suivants :

```
Index: 0   1               2      3      4       5      6         7
8
Token: INT IDENTIFIER(...) LPAREN RPAREN LBRACE RETURN NUMBER(2) SEMICOLON
RBRACE

       Au départ, tu es ici :
       ↓
       current = 0
```

À chaque action, le curseur peut se déplacer :

```
Après avoir lu INT :
       ↓
       current = 1
```

Pourquoi c'est nécessaire ?

Parce que tu **ne peux pas** lire tous les tokens d'un coup. Tu dois les lire **séquentiellement, un par un**.

Sans curseur, tu ne saurais pas :

- Quel token tu lis actuellement
- Si tu as fini de lire tous les tokens
- Quand tu dois arrêter

C'est comme lire un livre : tu dois savoir quelle page tu lis, sinon tu serais perdu.

Le curseur dans la boucle de parsing

```
Initialisation : current = 0 (premier token)

Boucle du parser :
├── Étape 1 : check(current) → "quel token à la position current ?"
├── Étape 2 : traiter le token
├── Étape 3 : advance() → current++ (passer au token suivant)
├── Étape 4 : vérifier si on a fini (current >= nombre total de tokens)
└── Si pas fini, retour à l'étape 1
```

Important : Le curseur ne diminue jamais

Une fois que tu avances, tu **ne reviens jamais en arrière**. Le curseur ne fait que croître.

```
Mauvais :
current = 5
advance() → current = 6
advance() → current = 5  ❌ NON ! Pas autorisé

Bon :
current = 5
advance() → current = 6
advance() → current = 7  ✅ Toujours avancer
```

Concept 2 : Regarder vs Avancer

Regarder (Peek) : Voir sans bouger

Qu'est-ce que c'est ?

Tu **observes le token courant** sans faire aucune action. Le curseur **ne change pas**.

Exemple détaillé de Peek

```
État initial :
current = 2
Tokens : INT IDENTIFIER LPAREN ← tu es ici RPAREN ...
                               ↓

peek() fait :
├── Regarde le token à la position current (index 2)
├── Le token est : LPAREN
├── Retourne : LPAREN
└── NE change pas current

État après peek() :
current = 2  ← Exactement pareil !
```

Cas d'usage de Peek

Tu utilises **peek()** quand tu veux **savoir ce qui vient** avant de **décider quoi faire**.

Exemple :

```
Je dois parser un statement. Les options sont :
- RETURN → statement de retour
- IF → statement de condition
- WHILE → statement de boucle
```

```
Je fais :
current = 5
peek() → "quel est le token à index 5 ?"
réponse → RETURN
action → donc je parse un return statement
```

Important : Tu ne fais **rien** avec le token tant que tu ne sais pas quoi en faire.

Avancer (Advance) : Passer au suivant

Qu'est-ce que c'est ?

Tu **consommes le token courant** et tu **passes au suivant**. Le curseur **augmente de 1**.

Exemple détaillé d'Advance

```
État initial :
current = 2
Tokens : INT IDENTIFIER LPAREN ← tu es ici RPAREN ...

advance() fait :
├── Tu as terminé avec le token à index 2
├── Augmente current de 1
├── current = current + 1 = 3
└── Maintenant tu es sur le token suivant

État après advance() :
current = 3
Tokens : INT IDENTIFIER LPAREN RPAREN ← tu es maintenant ici ...
```

Cas d'usage d'Advance

Tu utilises **advance()** quand tu as **traité un token** et que tu veux **passer au suivant**.

Exemple :

```
Je dois lire INT au début d'une fonction.

Étape 1 : peek() → "c'est INT ?"
          Oui !

Étape 2 : je traite INT (je le reconnais comme type)

Étape 3 : advance() → "OK, j'ai fini avec INT, passe au suivant"

Résultat : current = index du token suivant
```

La différence en résumé

Opération	Effet sur curseur	Utilité
peek()	Aucun changement	Regarder avant de décider
advance()	current++	Consommer un token et passer au suivant

Utilisation combinée de Peek et Advance

```
Tokens : INT IDENTIFIER LPAREN ...
Index:   0   1           2

current = 0

Étape 1 : peek() → regarde le token à index 0
         réponse → INT

Étape 2 : Je décide de traiter ce token INT

Étape 3 : advance() → j'ai terminé avec INT
         current = 1

Étape 4 : peek() → regarde le token à index 1
         réponse → IDENTIFIER

Étape 5 : advance() → j'ai terminé avec IDENTIFIER
         current = 2

Étape 6 : peek() → regarde le token à index 2
         réponse → LPAREN
```

Concept 3 : Vérifier (Check vs Match vs Expect)

Check : "Est-ce que c'est ce token ?" (observation simple)

Définition exacte :

Tu **examines le token courant** pour voir si c'est du type que tu attendais. **Aucune action**, juste une observation.

Exemple détaillé de Check

```
Situation :
current = 2
Tokens : INT IDENTIFIER LPAREN ...

check(LPAREN) fait :
├── Regarde le token à la position current (index 2)
├── Le token est : LPAREN
├── La question est : "est-ce LPAREN ?"
```

```
  ├── Réponse : OUI
  └── Retourne : true

  État après check() :
  current = 2  ← PAS CHANGÉ !
  Le curseur n'a pas bougé d'un poil.
```

Cas d'usage de Check

Tu utilises **check()** quand tu veux **prendre une décision** basée sur le token courant **SANS le consommer**.

Exemple concret :

```
  Je dois parser une statement. Elle peut être :
  - RETURN ... (si elle commence par RETURN)
  - IF ... (si elle commence par IF)
  - WHILE ... (si elle commence par WHILE)

  current = 5

  Étape 1 : check(RETURN) → "est-ce RETURN à index 5 ?"
           réponse → NON

  Étape 2 : check(IF) → "est-ce IF à index 5 ?"
           réponse → NON

  Étape 3 : check(WHILE) → "est-ce WHILE à index 5 ?"
           réponse → OUI !

  Action : Je sais que c'est un WHILE, donc j'appelle parseWhile()
```

Important : Le curseur ne change pas pendant ces vérifications. Tu essaies juste de savoir ce qui vient.

Match : "Est-ce que c'est ce token ? Si oui, avance." (observation + action)

Définition exacte :

C'est une **combinaison** de check() et advance(). Tu vérifes ET tu avanc dans la même opération.

Exemple détaillé de Match

```
  Situation :
  current = 2
  Tokens : INT IDENTIFIER LPAREN ...

  match(LPAREN) fait :
  ├── Étape 1 : Vérifier (comme check())
  │   ├── Regarde le token à index 2
  │   ├── Est-ce LPAREN ?
```

```
│   └── Réponse : OUI
│
├── Étape 2 : Puisque c'est bon, avancer
│   ├── current = current + 1
│   └── current = 3
│
└── Retourne : true

État après match(LPAREN) :
current = 3  ← CHANGÉ !
```

Exemple de Match qui échoue

```
Situation :
current = 2
Tokens : INT IDENTIFIER RETURN ...

match(LPAREN) fait :
├── Étape 1 : Vérifier (comme check())
│   ├── Regarde le token à index 2
│   ├── Est-ce LPAREN ?
│   └── Réponse : NON
│
├── Étape 2 : Puisque ce n'est pas bon, NE PAS avancer
│   └── current reste 2
│
└── Retourne : false

État après match(LPAREN) quand le token n'était pas LPAREN :
current = 2  ← INCHANGÉ !
```

Cas d'usage de Match

Tu utilises **match()** quand tu veux **vérifier ET consommer** un token dans la même action.

Exemple concret :

```
Je suis dans parseType(). Les types possibles sont INT, VOID, FLOAT.

current = 0
Tokens : INT ...

parseType() fait :
├── match(INT) ? → "est-ce INT et je l'avance ?"
│   ├── Oui ! Je consomme INT
│   ├── current = 1
│   └── Retourne la string "int"
│
└── Retour : "int"
```

```
Si c'était VOID :
├── match(INT) ? → "est-ce INT ?"
│   └── Non, current reste 0
├── match(VOID) ? → "est-ce VOID et je l'avance ?"
│   ├── Oui ! Je consomme VOID
│   ├── current = 1
│   └── Retourne la string "void"
```

Expect : "C'est OBLIGATOIRE" (vérification stricte)

Définition exacte :

Tu **exiges** que le token courant soit du type spécifié. Si ce n'est pas le cas, c'est une **ERREUR GRAVE** et tu arrêtes le parser.

Exemple détaillé d'Expect

```
Situation :
current = 2
Tokens : INT IDENTIFIER LPAREN ...

expect(LPAREN) fait :
├── Regarde le token à index 2
├── Est-ce LPAREN ?
├── Réponse : OUI
├── Avance : current = 3
└── Retourne : succès (pas d'erreur)

État après expect(LPAREN) réussi :
current = 3
```

Exemple d'Expect qui échoue

```
Situation :
current = 2
Tokens : INT IDENTIFIER RETURN ...

expect(LPAREN) fait :
├── Regarde le token à index 2
├── Est-ce LPAREN ?
├── Réponse : NON ! C'est RETURN
├── CRISE GRAVE !
│   ├── Lance une exception ParseException
│   ├── Message : "Expected '(' but found 'return'"
│   └── Arrête complètement le parser
└── LE PARSER S'ARRÊTE ICI
```

La différence critique entre Match et Expect

```
match(LPAREN) :
├─ "Est-ce LPAREN ?"
├─ Si oui → avance et retourne true
└─ Si non → NE fait rien et retourne false (peut continuer)

expect(LPAREN) :
├─ "Est-ce LPAREN ?"
├─ Si oui → avance et retourne normalement
└─ Si non → LANCE UNE ERREUR et ARRÊTE TOUT
```

En résumé : match() est "gentil" (retourne false), expect() est "strict" (lance une erreur).

Table comparative des trois fonctions

Fonction	Vérification ?	Avance ?	Si pas trouvé ?	Utilité
check()	Oui	Non	Retourne false	Juste regarder pour décider
match()	Oui	Si OK	Retourne false	Vérifier et consommer optionnellement
expect()	Oui	Si OK	**ERREUR**	Exiger que ce soit là

Quand utiliser chacune ?

Check() : Quand tu as plusieurs possibilités et tu dois choisir laquelle

```
Exemple : Un statement peut être RETURN, IF, ou WHILE

current = 5
if (check(RETURN)) → parseReturnStatement()
else if (check(IF)) → parseIfStatement()
else if (check(WHILE)) → parseWhileStatement()
else → erreur
```

Match() : Quand quelque chose est optionnel

```
Exemple : Dans une déclaration de fonction, les paramètres sont optionnels

current = 3
if (match(LPAREN)) {
    // Il y a des parenthèses, les traiter
} else {
    // Pas de parenthèses, c'est OK aussi
}
```

Expect() : Quand quelque chose est OBLIGATOIRE

```
Exemple : Après RETURN, il DOIT avoir une expression

expect(RETURN);  // Vérifie et avance
expression = parseExpression();  // Parse l'expression
expect(SEMICOLON);  // Il DOIT y avoir un semicolon
```

Exemple d'Expect avec messages clairs

Quand **expect()** échoue, le message d'erreur doit être **très informatif** :

```
Mauvais message :
"Parse error"
L'utilisateur ne sait pas quoi faire.

Bon message :
"Ligne 5, colonne 12 : Expected ';' but found 'return'"
L'utilisateur sait :
- Où : ligne 5, colonne 12
- Quoi : attendait un ';'
- Quoi trouvé : un token 'return'
```

Pourquoi Expect est crucial

Si tu **ignores** une erreur expect() et que tu continues le parsing :

```
Code source : int main() { return 2 ← MANQUE LE SEMICOLON
                                  }

expect(SEMICOLON) échoue.

Si tu ignores l'erreur :
├── Tu continues à parser
├── Prochaine fonction voit }
├── Elle interprète } comme une expression bizarre
└── AST complètement faux

Résultat : Le compilateur génère du code très étrange ou plante.
```

C'est pour ça que **expect()** arrête **immédiatement** : mieux vaut arrêter et dire à l'utilisateur "erreur ligne 5" que continuer et créer un AST cassé.

Concept 4 : Les règles de grammaire et les fonctions parser

Une grammaire = Des règles formelles

Une **grammaire** est un ensemble de **règles** qui décrivent **exactement** comment construire des structures valides.

Analogie : Une recette de gâteau est une grammaire. Elle dit :

- "D'abord, mélanger farine + œufs + sucre"
- "Puis, verser dans un moule"
- "Puis, cuire 30 minutes à 180°C"

Si tu fais quelque chose d'autre, ce n'est plus un gâteau valide.

Exemple de règle de grammaire

```
fonction = type IDENTIFIER LPAREN RPAREN LBRACE statement* RBRACE
```

Ça signifie **exactement** :

1. Un **type** (INT, VOID, FLOAT)
2. Suivi d'un **IDENTIFIER** (le nom)
3. Suivi d'une **parenthèse ouvrante**
4. Suivi d'une **parenthèse fermante**
5. Suivi d'une **accolade ouvrante**
6. Suivi de **zéro ou plusieurs statements**
7. Suivi d'une **accolade fermante**

Si l'un de ces éléments manque ou est dans le mauvais ordre, **ce n'est pas une fonction valide**.

Chaque règle de grammaire = Une fonction parser

Concept fondamental :

Chaque règle en grammaire devient une **fonction en code** qui :

1. **Vérifie** que le code respecte cette règle
2. **Consomme** les tokens nécessaires
3. **Construit** un nœud AST

```
Grammaire en EBNF :
function = type IDENTIFIER LPAREN RPAREN LBRACE statement* RBRACE

Code en C++ :
Function* parseFunction() { ... }
```

```
Grammaire :
type = INT | VOID | FLOAT
```

```
Code :
std::string parseType() { ... }
```

```
Grammaire :
statement = RETURN expression SEMICOLON | IF ...

Code :
Statement* parseStatement() { ... }
```

Qu'est-ce qu'une fonction parser doit EXACTEMENT faire ?

Chaque fonction parser doit accomplir **précisément ces 4 étapes** :

Étape 1 : Vérifier les préconditions

Avant de commencer, tu dois vérifier que les tokens correspondent à ce que tu t'attends.

```
Exemple : parseFunction()
├── Au démarrage, current = 0
├── Je vérifie : "est-ce que le token à index 0 est un TYPE ?"
└── Si non, c'est une erreur, arrête
```

Étape 2 : Traiter les éléments

Tu consommes les tokens en appelant d'autres fonctions ou en avançant.

```
Exemple : parseFunction()
├── Appelle parseType() → consomme le type
├── expect(IDENTIFIER) → consomme le nom
├── expect(LPAREN) → consomme la parenthèse
├── expect(RPAREN) → consomme la parenthèse fermante
├── expect(LBRACE) → consomme l'accolade
├── Boucle pour les statements
└── expect(RBRACE) → consomme l'accolade fermante
```

Étape 3 : Créer le nœud AST

Avec tous les éléments récoltés, tu crées un nœud qui les contient.

```
Exemple : parseFunction()
├── J'ai : return_type = "int"
├── J'ai : name = "main"
├── J'ai : body = [Return(...)]
└── Je crée : Function("int", "main", [Return(...)])
```

Étape 4 : Retourner le nœud

Tu retournes le nœud AST créé pour que la fonction parente l'utilise.

```
Exemple : parseFunction()
└── return new Function("int", "main", [Return(...)])
```

Pattern général avec détails

```
parseRule() fait :

ÉTAPE 1 : VÉRIFIER
├── Vérifier que le token courant est ce qu'on attend
├── Utiliser check() ou expect() pour vérifier
└── Si problème, lancer une exception

ÉTAPE 2 : TRAITER
├── Pour chaque élément de la règle :
│   ├── Si c'est un token simple (INT, LPAREN, etc.) :
│   │   └── expect() pour vérifier et avancer
│   │
│   ├── Si c'est une autre règle :
│   │   └── Appeler la fonction parser correspondante
│   │
│   ├── Si c'est une répétition (* ou +) :
│   │   └── Boucler en appelant la fonction parser
│   │
│   └── Stocker les résultats
│
ÉTAPE 3 : CRÉER LE NŒUD AST
├── Créer un nouvel objet du type approprié
├── Passer tous les éléments comme paramètres
└── Stocker le nœud

ÉTAPE 4 : RETOURNER
└── return nœud;
```

Exemple 1 en détail : parseType()

Grammaire :

```
type = INT | VOID | FLOAT
```

Ce que parseType() doit faire :

1. **Vérifier** quel type c'est

2. **Avancer** au token suivant
3. **Retourner** le type en tant que string

Trace d'exécution complète :

```
Tokens : INT IDENTIFIER ...
Index:   0   1

Appel à parseType() avec current = 0 :

ÉTAPE 1 : VÉRIFIER
├── check(INT) → "est-ce INT à la position 0 ?"
├── Regarde le token à index 0
├── C'est INT
└── Réponse : OUI

ÉTAPE 2 : TRAITER
├── Je sais que c'est INT
├── Je dois avancer pour passer à l'identifier suivant
├── match(INT) → vérifie et avance
├── current = 1 maintenant

ÉTAPE 3 : CRÉER LE NŒUD
├── J'ai le type : "int"
└── Pas d'objet à créer, juste une string

ÉTAPE 4 : RETOURNER
└── return "int"

État après parseType() :
├── current = 1 (avancé)
├── Résultat retourné : "int"
└── Prêt pour la prochaine fonction
```

Exemple 2 en détail : parseFunction()

Grammaire :

```
function = type IDENTIFIER LPAREN RPAREN LBRACE statement* RBRACE
```

Ce que parseFunction() doit faire :

1. Appeler parseType() pour le type
2. Vérifier et avancer sur IDENTIFIER
3. Vérifier et avancer sur LPAREN
4. Vérifier et avancer sur RPAREN
5. Vérifier et avancer sur LBRACE
6. Boucler sur parseStatement() pour tous les statements

7. Vérifier et avancer sur RBRACE
8. Créer un nœud Function
9. Retourner le nœud

Trace d'exécution extrêmement détaillée :

```
Tokens : INT IDENTIFIER("main") LPAREN RPAREN LBRACE RETURN NUMBER(2)
SEMICOLON RBRACE
Index:   0   1                  2      3      4       5      6         7
8

Appel à parseFunction() avec current = 0 :

═══════════════════════════════════════════════════════════
ÉTAPE 1 : VÉRIFIER (optional pour les règles simples)
═══════════════════════════════════════════════════════════
(Pas vraiment d'erreur possible ici, on suppose que c'est une function)

═══════════════════════════════════════════════════════════
ÉTAPE 2 : TRAITER - Élément par élément
═══════════════════════════════════════════════════════════

Élément 1 : type
├── Appelle parseType()
│   ├── À l'intérieur de parseType() :
│   │   ├── match(INT) → Oui, avance
│   │   ├── current = 1
│   │   └── return "int"
│   └── Retour de parseType() : "int"
├── Type stocké = "int"
├── current = 1

Élément 2 : IDENTIFIER
├── expect(IDENTIFIER)
│   ├── check(IDENTIFIER) → Oui, le token à index 1 est IDENTIFIER
│   ├── Avance
│   ├── current = 2
│   └── previous()->value = "main"
├── Nom stocké = "main"
├── current = 2

Élément 3 : LPAREN
├── expect(LPAREN)
│   ├── check(LPAREN) → Oui, le token à index 2 est LPAREN
│   ├── Avance
│   └── current = 3

Élément 4 : RPAREN
├── expect(RPAREN)
│   ├── check(RPAREN) → Oui, le token à index 3 est RPAREN
│   ├── Avance
│   └── current = 4
```

```
Élément 5 : LBRACE
├── expect(LBRACE)
│   ├── check(LBRACE) → Oui, le token à index 4 est LBRACE
│   ├── Avance
│   └── current = 5

Élément 6 : statement*
├── Boucle while (!check(RBRACE)) :
│   │
│   Itération 1 :
│   ├── check(RBRACE) → "est-ce RBRACE à index 5 ?"
│   ├── Non, c'est RETURN
│   ├── Appelle parseStatement()
│   │   ├── À l'intérieur de parseStatement() :
│   │   │   ├── check(RETURN) → Oui
│   │   │   ├── expect(RETURN) → Avance, current = 6
│   │   │   ├── Appelle parseExpression()
│   │   │   │   ├── parseExpression() voit NUMBER(2)
│   │   │   │   ├── Crée Constant(2)
│   │   │   │   ├── Avance, current = 7
│   │   │   │   └── return Constant(2)
│   │   │   ├── expect(SEMICOLON) → Avance, current = 8
│   │   │   ├── Crée Return(Constant(2))
│   │   │   └── return Return(Constant(2))
│   │   └── Retour de parseStatement() : Return(Constant(2))
│   ├── Ajoute à la liste des statements
│   ├── body = [Return(Constant(2))]
│   │
│   Condition de la boucle :
│   ├── check(RBRACE) → "est-ce RBRACE à index 8 ?"
│   ├── Oui, c'est RBRACE
│   └── Sort de la boucle

Élément 7 : RBRACE
├── expect(RBRACE)
│   ├── check(RBRACE) → Oui, le token à index 8 est RBRACE
│   ├── Avance
│   └── current = 9

═══════════════════════════════════════════════════════════
ÉTAPE 3 : CRÉER LE NŒUD
═══════════════════════════════════════════════════════════
├── Éléments disponibles :
│   ├── return_type = "int"
│   ├── name = "main"
│   └── body = [Return(Constant(2))]
│
├── Crée : new Function("int", "main", [Return(Constant(2))])
└── Stocke le nœud

═══════════════════════════════════════════════════════════
ÉTAPE 4 : RETOURNER
═══════════════════════════════════════════════════════════
```

```
└── return new Function("int", "main", [Return(Constant(2))])

État final après parseFunction() :
├── current = 9 (all tokens consumed)
├── Résultat : Function("int", "main", [Return(Constant(2))])
└── Prêt pour la fonction parente
```

Exemple 3 en détail : parseStatement()

Grammaire :

```
statement = RETURN expression SEMICOLON
          | IF LPAREN expression RPAREN statement
          | LBRACE statement* RBRACE
```

Ce que parseStatement() doit faire :

1. **Vérifier** quel type de statement c'est
2. **Appeler** la fonction parser appropriée
3. **Retourner** le nœud créé

Trace d'exécution :

```
Tokens : RETURN NUMBER(2) SEMICOLON
Index:   0      1         2

Appel à parseStatement() avec current = 0 :

────────────────────────────────────────────────────────────
ÉTAPE 1 : VÉRIFIER (détermine le type)
────────────────────────────────────────────────────────────
├── check(RETURN) → "est-ce RETURN à index 0 ?"
├── Oui !
└── Je sais que c'est un return statement

────────────────────────────────────────────────────────────
ÉTAPE 2 : TRAITER (pour un RETURN statement)
────────────────────────────────────────────────────────────
├── expect(RETURN) → Avance, current = 1
├── Appelle parseExpression()
│   ├── parseExpression() à current = 1
│   ├── Voit NUMBER(2)
│   ├── Crée Constant(2)
│   ├── Avance, current = 2
│   └── return Constant(2)
├── Réception : expression = Constant(2)
├── expect(SEMICOLON) → Avance, current = 3

────────────────────────────────────────────────────────────
```

```
ÉTAPE 3 : CRÉER LE NŒUD
────────────────────────────────────────────────────────────
├── Crée : new Return(Constant(2))
└── Stocke le nœud

────────────────────────────────────────────────────────────
ÉTAPE 4 : RETOURNER
────────────────────────────────────────────────────────────
└── return new Return(Constant(2))

État final :
├── current = 3
├── Résultat : Return(Constant(2))
└── Prêt pour la fonction parente
```

Patterns communs des fonctions parser

Pattern 1 : Séquence (A puis B puis C)

```
Grammaire : rule = A B C

Exemple : function = type IDENTIFIER LBRACE statement* RBRACE

parseRule() fait :
├── Appelle parseA() → récupère A
├── Appelle parseB() → récupère B
├── Appelle parseC() → récupère C
├── Crée un nœud avec A, B, C
└── return nœud
```

Pattern 2 : Alternatives (A ou B ou C)

```
Grammaire : rule = A | B | C

Exemple : type = INT | VOID | FLOAT

parseRule() fait :
├── if (check(TOKEN_A)) {
│   └── return parseA()
├── } else if (check(TOKEN_B)) {
│   └── return parseB()
├── } else if (check(TOKEN_C)) {
│   └── return parseC()
└── } else throw error
```

*Pattern 3 : Répétitions zéro ou plus (item)**

```
Grammaire : rule = item*

Exemple : function = ... LBRACE statement* RBRACE

parseRule() fait :
├── Crée une liste vide : items = []
├── while (!check(STOP_TOKEN)) {
│   ├── Appelle parseItem()
│   ├── Ajoute à la liste : items.push_back(item)
│   └── continue la boucle
├── }
└── return items (peut être vide)
```

Pattern 4 : Répétitions un ou plus (item+)

```
Grammaire : rule = item+

parseRule() fait :
├── Doit avoir au moins un item
├── Appelle parseItem() UNE FOIS
├── Ajoute à la liste
├── while (peut continuer) {
│   ├── Appelle parseItem()
│   ├── Ajoute à la liste
│   └── continue
├── }
└── return items (au moins 1)
```

Pattern 5 : Optionnel (item?)

```
Grammaire : rule = item?

Exemple : if_stmt = IF condition statement [ELSE statement]

parseRule() fait :
├── if (check(TOKEN_ITEM)) {
│   ├── item = parseItem()
│   └── avance
├── } else {
│   └── item = null ou valeur par défaut
├── }
└── return item ou (item, default_value)
```

Important : Le gestion du curseur dans les fonctions parser

Règle d'or : Après chaque fonction parser, le curseur **DOIT** être avancé pour refléter tous les tokens consommés.

```
Avant parseFunction() : current = 0 (at INT)
                        Tokens : INT IDENTIFIER LPAREN ...

Après parseFunction() :  current = 9 (après le RBRACE)
                         Tous les tokens de la fonction ont été consommés
```

Si une fonction **ne consomme pas correctement les tokens** :

```
Mauvais :
parseFunction() :
├── Lit les tokens
├── Mais oublie d'avancer le curseur
└── current reste à sa valeur d'avant

Résultat : La fonction suivante voit les mêmes tokens
           Et parse du code complètement faux
```

Erreurs critiques dans les fonctions parser

Erreur 1 : Ne pas avancer le curseur

```
parseType() fait :
├── Voit INT
├── Retourne "int"
└── OUBLIE d'avancer

Résultat : La fonction suivante voit toujours INT
```

Erreur 2 : Utiliser check() quand il faut expect()

```
Mauvais :
parseFunction() fait :
├── check(LPAREN) → faux
├── Continue comme si de rien n'était
└── Parse du code invalide

Bon :
parseFunction() fait :
├── expect(LPAREN) → faux
├── Lance une exception
└── S'arrête et informe l'utilisateur
```

Erreur 3 : Oublier de boucler pour les répétitions

```
Grammaire : statement*

Mauvais :
parseStatements() fait :
├── Appelle parseStatement() UNE FOIS
└── Retourne une liste avec UN seul statement

Bon :
parseStatements() fait :
├── Boucle while il y a des statements
└── Retourne une liste avec TOUS les statements
```

Erreur 4 : Mauvaise gestion des alternatives

```
Grammaire : statement = RETURN ... | IF ... | WHILE ...

Mauvais :
parseStatement() fait :
├── check(RETURN) → parse return
├── check(IF) → AUSSI parse IF (peut être fait deux fois !)
└── check(WHILE) → AUSSI parse WHILE

Bon :
parseStatement() fait :
├── if (check(RETURN)) return parseReturn()
├── else if (check(IF)) return parseIf()
├── else if (check(WHILE)) return parseWhile()
└── else throw error
```

Concept 5 : Parsing récursif descendant

Qu'est-ce que c'est exactement ?

Parsing récursif descendant est une **stratégie** pour implémenter un parser. L'idée est simple :

Tu **descends l'arbre de grammaire en appel recursifs**, du plus haut niveau (le programme entier) jusqu'aux plus bas niveaux (les tokens individuels).

Pourquoi "descendant" ?

Parce que tu **commences par le haut** (règle générale) et tu **descends vers le bas** (détails spécifiques).

```
HAUT (général) :
parseProgram()
    ↓ appelle
parseFunction()
    ↓ appelle
```

```
parseBlock()
    ↓ appelle
parseStatement()
    ↓ appelle
parseExpression()
    ↓ appelle
parseTerm()
    ↓ appelle
parseFactor()
BAS (spécifique) : les tokens individuels
```

Pourquoi "récursif" ?

Parce que les fonctions **s'appellent elles-mêmes** ou **s'appellent l'une l'autre en créant une chaîne d'appels**.

```
parseExpression() appelle parseTerm()
parseTerm() appelle parseFactor()
parseFactor() appelle parseExpression() (récursion !)

Pour : (2 + 3) * 4
```

Exemple de récursion complète

```
Code à parser : int main(void) { return (2 + 3); }

parseProgram() appelle
    └── parseFunction() appelle
        ├── parseType() retourne "int"
        └── parseBlock() appelle
            └── parseStatement() appelle
                └── parseExpression() appelle
                    └── parseTerm() appelle
                        └── parseFactor() appelle
                            └── parseExpression() (appel récursif pour les
parenthèses) appelle
                                └── parseTerm() appelle
                                    └── ... (continue jusqu'aux tokens)
```

Comment la récursion remonte

L'information **remonte** à travers les appels de fonction :

```
parseFactor() crée Constant(2)
    ↑ retourne à
parseTerm() qui combine avec l'opérateur
```

```
    ↑ retourne à
parseExpression() qui combine
    ↑ retourne à
parseStatement() qui crée Return(...)
    ↑ retourne à
parseBlock() qui l'ajoute à une liste
    ↑ retourne à
parseFunction() qui en fait le corps
    ↑ retourne à
parseProgram() qui crée Program([Function(...)])
```

Chaque niveau **construit un nœud** et le **passe au niveau supérieur**.

Concept 6 : Construire l'AST pendant le parsing

Quand construit-on l'AST ?

Simultanément avec le parsing ! Ce n'est **pas** deux étapes séparées.

À chaque appel de fonction parser, tu :

1. **Traites** les tokens
2. **Construis** un nœud AST
3. **Retournes** le nœud

Pourquoi pendant et pas après ?

Parce que c'est **beaucoup plus simple** !

- Tu avances dans les tokens
- En même temps, tu construis la structure
- Quand tu as fini, tu as directement un AST complet

Si tu parsais **d'abord** puis construisais **après**, tu devrais :

- Parser une fois
- Stocker tous les tokens intermédiaires
- Reparcourir ensuite pour construire l'AST
- Double travail !

Exemple d'AST construit pendant le parsing

```
Code : int main(void) { return 2; }

Tokens générés : INT IDENTIFIER("main") LPAREN RPAREN LBRACE RETURN
NUMBER(2) SEMICOLON RBRACE

Parsing ET construction simultanée :

parseProgram() est appelé
```

```
├── Boucle : il y a du code
│   ├── Appelle parseFunction()
│   │   ├── Appelle parseType()
│   │   │   └── Lit INT, RETOURNE String("int")
│   │   ├── Lit IDENTIFIER("main"), RETOURNE "main"
│   │   ├── Lit LPAREN et RPAREN
│   │   ├── Appelle parseBlock()
│   │   │   ├── Appelle parseStatement()
│   │   │   │   ├── Appelle parseExpression()
│   │   │   │   │   └── RETOURNE Constant(2)
│   │   │   │   └── RETOURNE Return(Constant(2))
│   │   │   └── RETOURNE [Return(Constant(2))]
│   │   └── CRÉE ET RETOURNE Function("int", "main",
[Return(Constant(2))])
│   │
│   └── AJOUTE Function à la liste
│
└── CRÉE ET RETOURNE Program([Function(...)])

Résultat FINAL : AST complet !

Program
└── functions: [
    Function(
        return_type: "int",
        name: "main",
        body: [
            Return(
                value: Constant(2)
            )
        ]
    )
]
```

Chaque niveau crée son nœud

```
Level 1 (haut) : parseProgram()
├── Crée : Program([...])

Level 2 : parseFunction()
├── Crée : Function(...)

Level 3 : parseStatement()
├── Crée : Return(...)

Level 4 : parseExpression()
├── Crée : Constant(...)

Level 5 (bas) : Tokens individuels
├── INT, IDENTIFIER, NUMBER, etc.
```

Chaque niveau crée son objet et le passe au niveau au-dessus.

(Document continué dans la partie suivante...)

Concept 7 : Vérifier la syntaxe (qu'elle est valide)

Qu'est-ce que ça signifie concrètement ?

Tu **vérifies** que les tokens **respectent exactement la grammaire** qu'tu as définie.

Exemple : Une fonction sans type est invalide

```
Code valide :
int main() { ... }

Code invalide :
main() { ... }  ← Pas de type ! Erreur de syntaxe

Grammaire : function = type IDENTIFIER LPAREN ...

Le parser fait :
├── Cherche un type
├── Voit IDENTIFIER au lieu d'un type
├── Erreur ! Ce n'est pas une fonction valide
```

Concept 8 : Les alternatives (OR / |)

Définition

Parfois, une règle a **plusieurs formes possibles**. Tu dois **choisir laquelle** en vérifiant les tokens.

Exemple avec détails

```
Grammaire :
statement = RETURN expression SEMICOLON
          | IF LPAREN expression RPAREN statement
          | WHILE LPAREN expression RPAREN statement

Tokens : IF ...

parseStatement() fait :
├── check(RETURN) → "est-ce RETURN ?"
├── Non
├── check(IF) → "est-ce IF ?"
├── OUI !
├── Appelle parseIfStatement()
└── Retourne le résultat
```

Important : L'ordre compte !

Si tu as deux règles qui commencent pareil, **l'ordre de vérification** détermine laquelle est choisie.

```
Mauvais ordre :
├── check(IDENTIFIER) → true pour "x = 5" ET pour "x + 5"
└── Peut parser les deux comme assignment

Bon ordre :
├── Mettre les règles plus spécifiques en premier
├── Puis les moins spécifiques
└── Ça évite les ambiguités
```

Concept 9 : Les répétitions (*, +)

Zéro ou plus (*)

```
Grammaire : statement*

Signifie : Il peut y avoir 0, 1, 2, ... statements
```

Un ou plus (+)

```
Grammaire : parameter+

Signifie : Il DOIT avoir au moins 1 paramètre
```

Comment implémenter les répétitions

```
Pour A* :
├── Boucle while il y a des A
│   ├── Parse un A
│   ├── Ajoute à la liste
│   └── continue
└── Retourne la liste (peut être vide)

Pour A+ :
├── Parse au moins un A
├── Ajoute à la liste
├── Boucle while il y en a d'autres
│   ├── Parse un A
│   ├── Ajoute à la liste
└── Retourne la liste (min 1)
```

Concept 10 : La précédence des opérateurs

Le problème exact

```
Expression : 2 + 3 * 4

Est-ce (2 + 3) * 4 = 20 ?
Ou     2 + (3 * 4) = 14 ?

La réponse : 14 ! Parce que * a plus de priorité que +
```

La solution : Niveaux de grammaire

```
expression = additive
additive = multiplicative (PLUS multiplicative)*
multiplicative = primary (STAR primary)*
primary = NUMBER | (expression)
```

Pourquoi ça marche ?

```
Pour 2 + 3 * 4 :

expression appelle additive
    ├── Appelle multiplicative
    │   ├── Appelle primary → 2
    │   └── Pas de STAR, retourne 2
    ├── Voit PLUS
    ├── Appelle multiplicative (pour le côté droit)
    │   ├── Appelle primary → 3
    │   ├── Voit STAR
    │   ├── Appelle primary → 4
    │   └── Retourne 3 * 4 = 12
    └── Retourne 2 + 12 = 14
```

Les multiplications sont **évaluées en premier** parce qu'elles sont au **plus bas niveau** de la hiérarchie.

Concept 11 : Les erreurs et la récupération

Types d'erreurs

Erreur syntaxique : Le code ne respecte pas la grammaire

```
return ;          ← Pas d'expression
return 2          ← Pas de semicolon
int main(void { } ← Parenthèse manquante
```

Comment détecter

Avec **expect()** : C'est **obligatoire**, donc si c'est absent, c'est une erreur.

```
expect(SEMICOLON) :
├── Voit SEMICOLON → OK
└── Voit autre chose → ERREUR
```

Messages d'erreur utiles

```
Mauvais :
"Error"

Bon :
"Erreur ligne 5, colonne 12 : expected ';' but found 'return'"

Meilleur :
"
Line 5, column 12: Syntax error
  int main(void) { return 2
                             ^
Expected ';' but found 'return'
"
```

Concept 12 : L'ordre des vérifications

Pourquoi l'ordre compte

```
Grammaire :
statement = RETURN ...
          | identifier = ...

Tokens : IDENTIFIER ...

Si je vérifie IDENTIFIER en premier :
├── match(IDENTIFIER) → true
└── Je parse comme assignment

Si je vérifie RETURN d'abord :
├── check(RETURN) → false
├── check(IDENTIFIER) → true
```

```
└── Je parse comme assignment

Dans ce cas, l'ordre ne change rien, mais dans d'autres cas si !
```

Concept 13 : Remonter les résultats

Chaque fonction retourne quelque chose

```
parseFunction() retourne Function(...)
    ↑
parseProgram() utilise ce Function(...) et l'ajoute à sa liste
    ↓
parseProgram() retourne Program([Function(...)])
```

L'information **remonte** à travers les appels.

Résumé complet des concepts

#	Concept	Rôle
1	**Curseur (current)**	Suivre quel token tu lis
2	**Peek / Advance**	Regarder vs consommer
3	**Check / Match / Expect**	Vérifier les tokens avec différentes stratégies
4	**Règles et fonctions parser**	Chaque règle = une fonction
5	**Parsing récursif descendant**	Descendre l'arbre en appels recursifs
6	**Construire l'AST**	Créer les nœuds pendant le parsing
7	**Vérifier la syntaxe**	Rejeter le code invalide
8	**Alternatives (OR)**	Gérer plusieurs possibilités
9	**Répétitions (*, +)**	Boucler pour les listes
10	**Précédence**	Respecter l'ordre des opérateurs
11	**Erreurs**	Détecter et signaler les problèmes
12	**Ordre des vérifications**	Première correspondance gagne
13	**Remontée des résultats**	Chaque fonction retourne un nœud

---

Ce que tu dois absolument comprendre

1. **Le parser navigue token par token** : Il a un curseur qui avance à chaque token traité.

2. **Check vs Match vs Expect** :
    - check() = juste regarder
    - match() = regarder et consommer si OK
    - expect() = consommer ou erreur
3. **Chaque règle de grammaire = une fonction** : La grammaire te dit exactement quelles fonctions créer.
4. **La récursion descend dans la structure** : parseProgram() appelle parseFunction() qui appelle parseStatement(), etc.
5. **L'AST est construit pendant le parsing** : Pas avant, pas après, **pendant**.
6. **Les erreurs arrêtent immédiatement** : Pas de continuation silencieuse.
7. **La précédence vient de la structure de grammaire** : Pas d'une table séparée.
8. **Chaque fonction avance le curseur** : C'est crucial pour que le parsing continue correctement.